

Unit-2

LLM

Dr. A N Ramya Shree
Dept. of CSE(AI&ML)

Unit-2 Syllabus

LLM, Text Representation and GPT Models:

Large Language Models: Introduction to Large Language Models (LLMs), What Are LLMs, Applications of LLMs, How LLMs See the World: Hallucinations, How LLMs Process Text, LLM Tokenization, How Tokenizers Prepare Inputs to the Language Model, How Does the Tokenizer Break Down Text, Word vs. Subword vs. Character vs. Byte Tokens, Comparing Trained LLM Tokenizers, Tokenizer Properties, Token Embeddings, Text Embeddings, Word Embedding.



LLM

A Large Language Model (LLM) is a neural network trained on a very large collection of text data and containing a very large number of parameters.

LLMs learn relationships between tokens and can perform tasks such as:

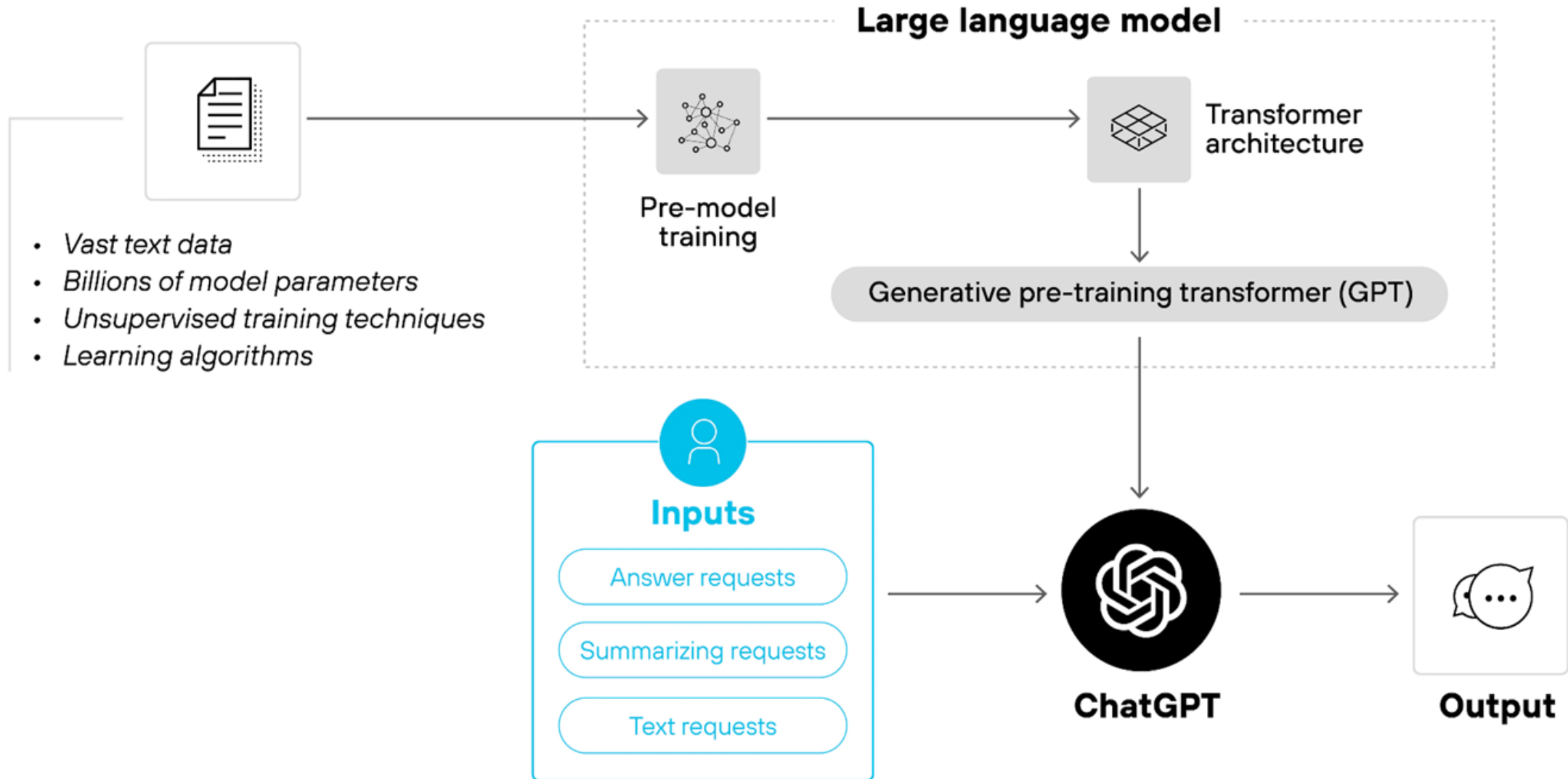
- Text generation
- Question answering
- Summarization
- Translation
- Classification
- Code generation
- Information extraction
- Conversational interaction



A language model is a type of artificial intelligence system that is trained to understand and generate human-like text. It learns the structure, grammar, and semantics of a language by processing vast amounts of textual data. The primary goal of a language model is to predict the probability of the next word or sequence of words in a given context.



Large language model (LLM)



An LLM does **not simply store a dictionary of sentences**.

- Instead, during training it learns numerical parameters that capture patterns and relationships in language.

Why are LLMs Called "Large"?

The word large generally refers to several aspects:

1. Large training datasets-Models are trained on enormous collections of text.
2. Large number of parameters-Parameters are numerical values learned during training.



- **Small (8B–13B parameters):** basic arithmetic, code debugging, reading comprehension, and basic language operations (e.g. creating stories, essays, poetry).
- **Medium (64B–175B parameters):** language puzzles, understanding and solving college admissions tests, physical intuition, logical deduction, or understanding metaphors.
- **Large (70B–540B parameters):** geometric shapes, phonetic alphabet, causality, elementary mathematics, and code explanation. • **Extra-large(1teraparameter):** spatial reasoning, creativity, app construction , picture handling, and thematic test performance.



Large computational requirements:

- Training modern LLMs requires substantial:
- GPU/TPU computation
- Memory
- Storage
- Training time

Large vocabulary: LLMs can process a very large number of tokens representing different languages and text patterns.



Types of Tokenization

- In the context of NLP tasks, *tokenization* refers to the way in which a text segment is represented as a sequence of vocabulary items, usually called *tokens*
- *Tokenization* depends on the level of granularity desired in a task.



- As an LM predictor, BERT's goal is to “fill in the blank,” depending on the context.

“John bought a _____ at the corner store.”

- An LM could complete this sentence by saying that the word “vehicle” would fill in the blank 20% of the time and the word “eggs” 80% of the time.



How Does an LLM Learn?

The capital of France is ____

The expected answer is: **Paris**

During training:

Input: The capital of France is

Expected: Paris

The model initially makes a prediction.

Suppose:

Paris → 0.20

London → 0.15

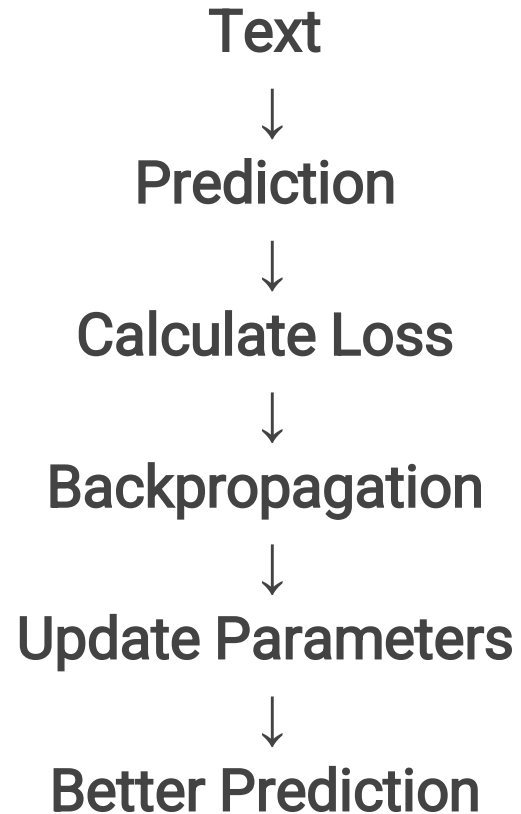
Rome → 0.10

Berlin → 0.05

The model calculates a loss between the prediction and the correct answer.

The parameters are then updated using backpropagation and gradient descent.

This process is repeated over enormous amounts of training data



Applications of LLMs

- Text Generation
- Summarization
- Question Answering
- Translation
- Code Generation
- Sentiment Analysis
- Information Extraction

How LLMs "See" the World

- LLMs do not see the world in the same way humans do.
- They primarily process numerical representations of tokens.
- An LLM does not directly experience reality.
- It learns patterns from the data it was trained on.

Human:

"The dog is running."

LLM:

"The" → token ID

"dog" → token ID

"is" → token ID

"running" → token ID



Numerical vectors



Transformer computations

Hallucinations

What is an LLM hallucination?

An LLM hallucination occurs when a model generates information that appears plausible but is incorrect, unsupported, or fabricated.

User:

Who invented the XYZ algorithm?

LLM:

The XYZ algorithm was invented by Professor ABC in 1987.

If the person or event does not actually exist, the model has hallucinated.

Why Do Hallucinations Occur?

- One important reason is that a generative language model is fundamentally trained to predict likely tokens.
- It is not automatically a fact-checking system.

The capital of France is

The model has strong statistical evidence for: Paris

But for an obscure question:

The winner of the 1873 XYZ competition was

the model may still generate a plausible continuation

Fluent language does not guarantee factual correctness.

Causes of Hallucination

- Insufficient training information
- Ambiguous prompts
- Outdated knowledge
- Conflicting training information
- Poorly specified questions
- Model tendency to generate plausible continuations
- Lack of external verification

Reducing Hallucinations

Better Prompt

+

Grounded Information

+

Retrieval-Augmented Generation

+

Tool Use

+

Fact Verification

+

Human Review

How LLMs Process Text?

Computers do not directly process words as humans do.

Consider: I love machine learning.

The LLM needs to convert this into numerical information.

Raw Text



Tokenization



Token IDs



Embedding



Transformer



Output probabilities

Tokenization is the process of dividing text into smaller units called **tokens**.

I love machine learning.

["I", "love", "machine", "learning", "."]

Modern LLMs generally use **subword tokenization**, so the result may be different.

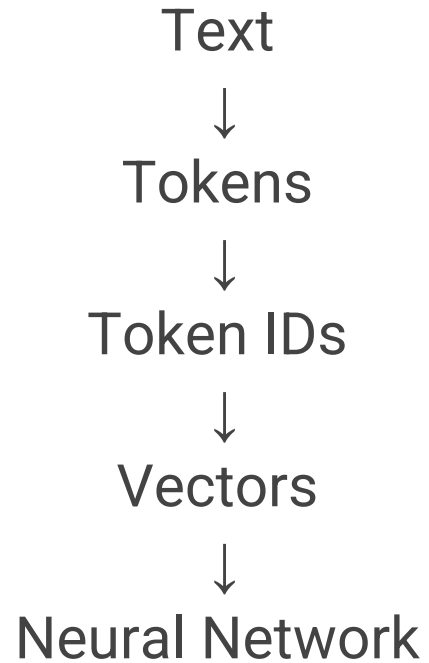
Unhappiness

["un", "happiness"]

["un", "happi", "ness"]

Why Do We Need Tokenization?

Computers need numerical input.



"I love AI"



["I", "love", "AI"]



[45, 982, 721]



Embedding vectors



Transformer

```
text = "I love machine learning."  
tokens = text.split()  
print(tokens)
```

```
['I', 'love', 'machine', 'learning.']
```



```
import re
text = "I love machine learning."
tokens = re.findall(r"\w+|[\^\\w\\s]", text)
print(tokens)
```

```
['I', 'love', 'machine', 'learning', '.']
```

Suppose we wish to *tokenize* the sentence:

“We want a complete pre-report”

the usual types of *tokenization* would be as follows:

- **Character-level *tokenization*:** the vocabulary V is the alphabet of the language (i.e., English) plus punctuations. In the example sentence, we would obtain a sequence of length 31: ['Q', 'u', ' ', ..., ...]. Usually, this type of *tokenization* tends to produce very long sequences.
- **Word-level *tokenization*:** the vocabulary V is set of all the words of the language in question, plus punctuations. In the example sentence, we would get a sequence of length 4: ['We want', 'a', 'pre-report', ...]. Usually, this type of *tokenization* requires a very large vocabulary and cannot handle new words at test time.
- **Subword *tokenization*:** the vocabulary V is a set of common word segments such as ‘pre,’ ‘mos,’ ‘pre.’ Common words such as ‘a’ are usually *tokenized*, and individual characters are also included in V , to ensure that all words can be expressed.

Word-Level Tokenization

In word-level tokenization, each word becomes a token.

I love AI

["I", "love", "AI"]

Advantages

- Easy to understand
- Tokens have meaningful linguistic units

Disadvantage

- play
- playing
- played
- player
- playful

Each can become a separate token.

This can produce a very large vocabulary.

BERT (Bidirectional Encoder Representations from Transformers)

- NLP models based on deep learning techniques require very large amounts of data to improve their performance when trained on millions or billions of annotated training examples.
- To help bridge this gap, several techniques have been developed to train general-purpose language representation models, better known as “pre-training,” which uses large corpora of unannotated text.
- These pre-trained models can be tuned to smaller task-specific datasets: question-answer systems, sentiment analysis, text prediction, text generation, etc.
- This approach shows significant improvements in accuracy compared to training on smaller task-specific datasets from scratch.

One of the models addressing these difficulties in pre-training for NLP tasks is called BERT (Devlin et al., 2019).

- BERT is a model that is trained bidirectionally for prediction tasks. This means that we can now have a deeper sense of context and language flow compared to unidirectional LMs (Devlin et al., 2019).
- BERT is based on a basic *transformer*, consisting only of *encoders*, to read text input, and a *decoder*, to produce a prediction for a task. This *transformer* applies attention mechanisms to understand the relationships between all words in a sentence, regardless of their respective positions.
- Generally, BERT can be used to extract high-quality language *features* from a corpus or to fit these models to a specific task (e.g., question answering) with proprietary data in order to produce good predictions.

Character-Level Tokenization

Each character becomes a token.

Advantages

- Very small vocabulary
- Can handle unknown words
- Works well with unusual words

Disadvantages

- A long sequence of characters is required to represent normal text.

Subword Tokenization

Subword tokenization divides words into frequently occurring pieces.

un + happi + ness

- The exact segmentation depends on the tokenizer.
- Handles unknown or rare words
- Vocabulary is manageable
- More efficient than character-level tokenization
- Can represent related words using common pieces

Modern LLM tokenizers commonly use subword/byte-level approaches

Byte-Level Tokenization

It starts from bytes rather than complete words.

- The text is represented using byte values and then tokenized into learned units.
- This is useful because bytes provide a mechanism for representing essentially arbitrary text.

It can help with:

- Unknown words
- Misspellings
- Special symbols
- Multiple languages
- Emojis

Method	Example: playing	Vocabulary	Sequence Length
Word	playing	Large	Short
Character	p l a y i n g	Very small	Long
Subword	play + ing	Medium	Medium
Byte	Byte/subunits	Small/base-level	Variable

```
pip install transformers
from transformers import AutoTokenizer
tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
text = "I love machine learning."
tokens = tokenizer.tokenize(text)
print(tokens)
```

nn.Linear

nn.Embedding

nn.Dropout

nn.Sequential

Input and target are shifted by one token.

tokenized text is:

I love machine learning

Token IDs (sample)

[10, 20, 30, 40]

For training:

Input: [10, 20, 30]

Target: [20, 30, 40]

So the model learns:

10 → 20

10 20 → 30

10 20 30 → 40

for i in range(0, len(token_ids) - max_length, stride):

max_length = 4

stride = 2

tokens are:

[1 2 3 4 5 6 7 8 9 10]

The windows will approximately be:

Window 1:

[1 2 3 4]

Window 2:

[3 4 5 6]

Window 3:

[5 6 7 8]

Window 4:

[7 8 9 10]

The windows overlap-a sliding window.

```
input_chunk = token_ids[i:i + max_length]
```

```
target_chunk = token_ids[i + 1:i + max_length + 1]
```

```
max_length=4
```

Input:

10 20 30 40

Target:

20 30 40 50

batch_size = 4

max_length = 256

4 - number of sequences

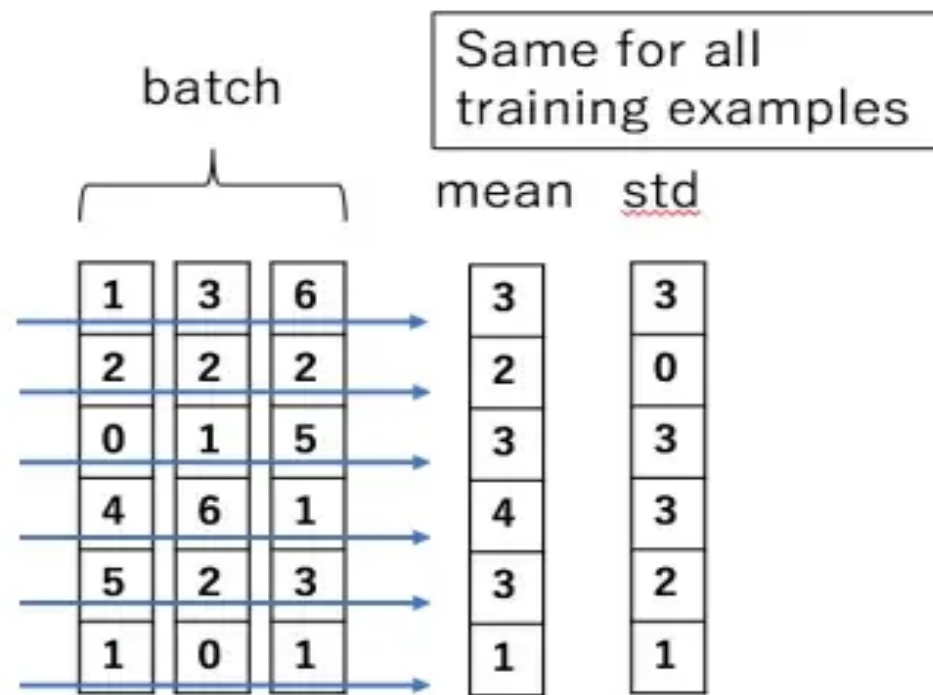
256 - tokens in each sequence

Input embedding + Query+ Key+ Value

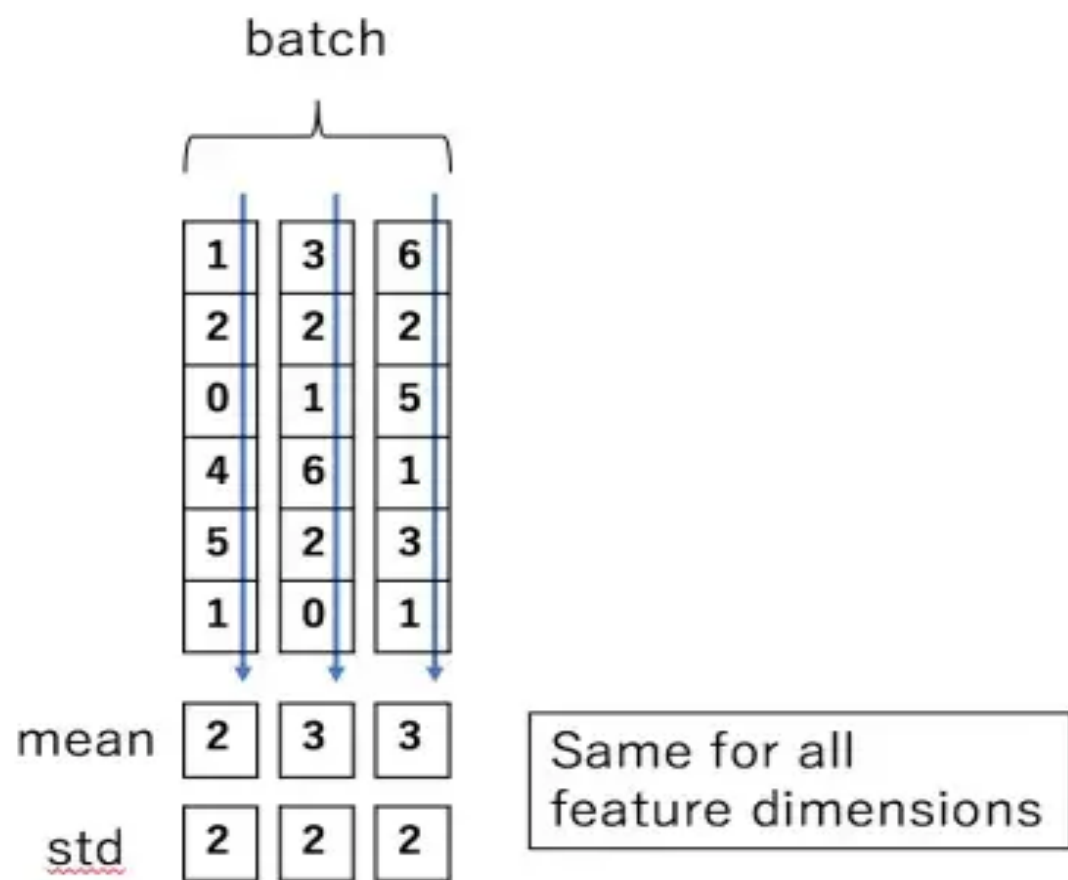
Attention weights $\times$ Value vectors = Context vector

Layer normalization helps keep the activations numerically stable.

Batch Normalization



Layer Normalization



Causal mask

This is extremely important for GPT.

The model must not look into the future.

I love machine learning

I	0	1	1	1
---	---	---	---	---

love	0	0	1	1
------	---	---	---	---

machine	0	0	0	1
---------	---	---	---	---

learning	0	0	0	0
----------	---	---	---	---

Gaussian Error Linear Unit activation function

GELU is commonly used in Transformer architectures.

It introduces non-linearity into the network.

$$GELU(x) = 0.5x \left[1 + \tanh \left(\sqrt{\frac{2}{\pi}} (x + 0.044715x^3) \right) \right]$$

Leaky ReLU

Leaky ReLU allows a small amount of negative input to pass through.

$$f(x) = \begin{cases} x & x \geq 0 \\ \alpha x & x < 0 \end{cases}$$

where α is a small positive value, commonly **0.01**.

x	ReLU	Leaky ReLU
-3	0	-0.03
-2	0	-0.02
-1	0	-0.01
0	0	0
1	1	1
2	2	2
3	3	3

Advantages of GELU

Smooth activation: GELU produces a continuous, differentiable curve with no sharp transitions, which gives the optimizer cleaner information to work with at every step.

Better gradient flow: GELU doesn't zero out negative inputs, so gradients can still propagate through neurons that receive negative values. This reduces the risk of neurons going dead during training.

Better performance in deep models: In deep architectures like transformers, the cumulative effect of smoother gradients tends to translate into better training results compared to simpler activation functions.

Limitations of GELU

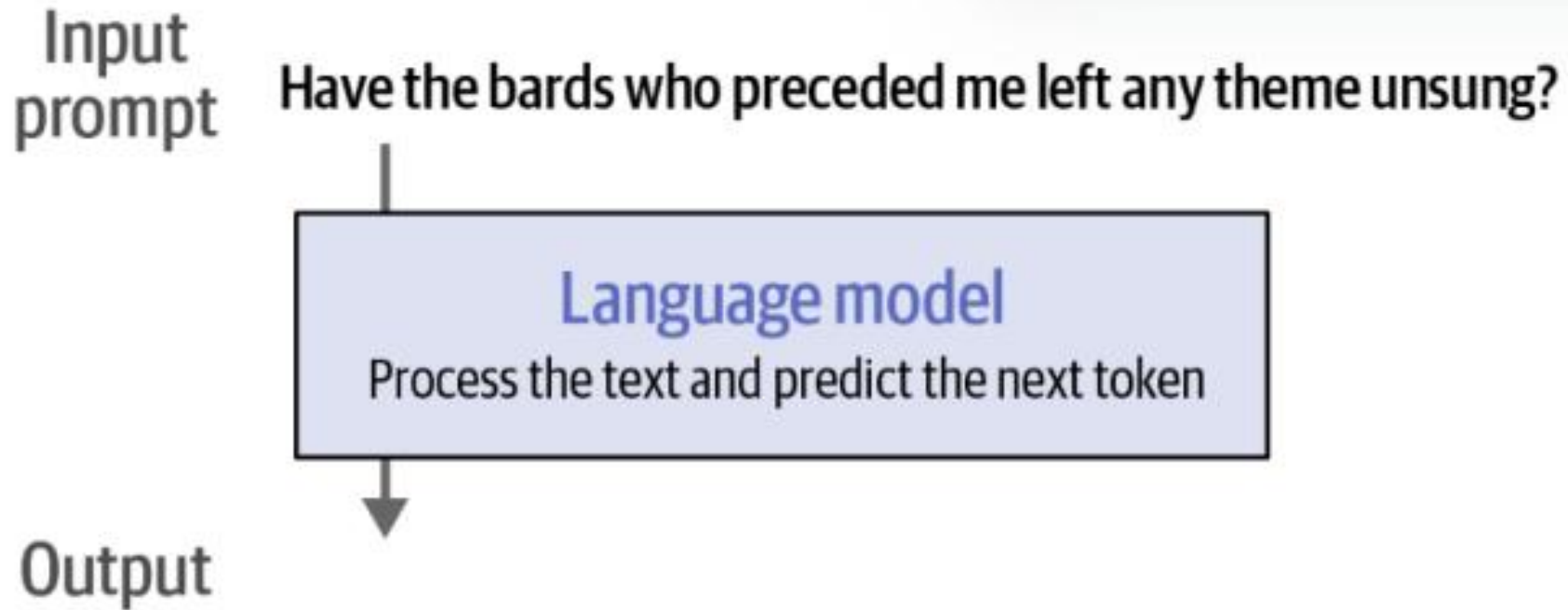
GELU isn't the right choice for every situation. Here are a couple of limitations you must be aware of:

- **More expensive to compute than ReLU:** GELU involves either an error function or a tanh based approximation, both of which cost more than ReLU's simple threshold operation. In large models with many layers, this can add up.
- **Less intuitive:** Functions like ReLU are easy to reason about - positive values pass, negative values don't. GELU's probabilistic scaling is harder to interpret.
- **Not always necessary:** For shallow networks or simpler tasks, GELU doesn't offer meaningful advantages. ReLU or Leaky ReLU will often perform just as well at a lower computational cost.

GELU vs Other Activation Functions



How Tokenizers Prepare the Inputs to the Language Model



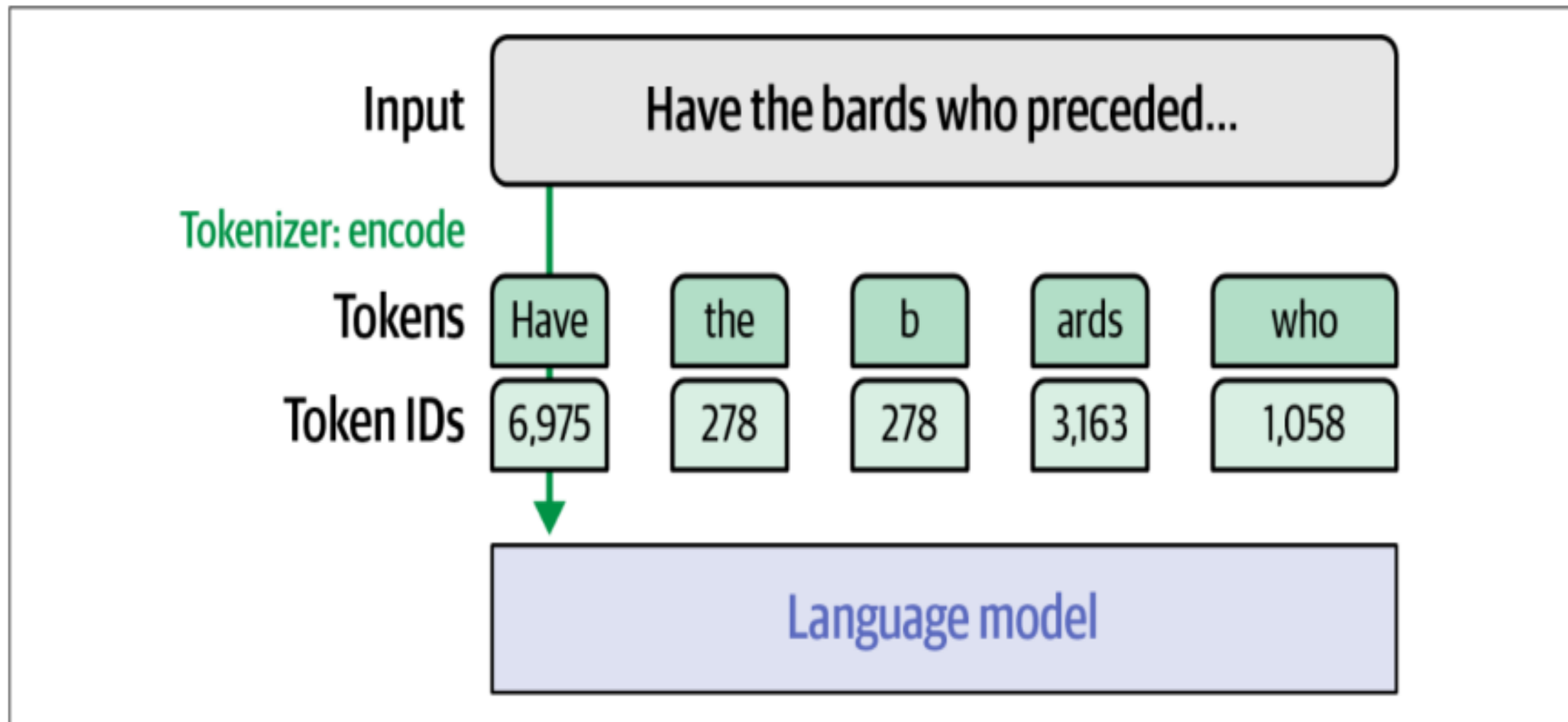


Figure 2-4. A tokenizer processes the input prompt and prepares the actual input into the language model: a list of token IDs. The specific token IDs in the figure are just demonstrative.

The first token is ID 1 (<s>), a special token indicating the beginning of the text.

- Some tokens are complete words .
- Some tokens are parts of words .
- Punctuation characters are their own token.

How Does the Tokenizer Break Down Text?

There are three major factors that dictate how a tokenizer breaks down an input prompt.

First, at model design time, the creator of the model chooses a tokenization method.

- Popular methods include byte pair encoding (BPE) (widely used by GPT models) and WordPiece (used by BERT).
- These methods are similar in that they aim to optimize an efficient set of tokens to represent a text dataset, but they arrive at it in different ways.

- **Second**, after choosing the method, we need to make a number of tokenizer design choices like vocabulary size and what special tokens to use.
- **Third**, the tokenizer needs to be trained on a specific dataset to establish the best vocabulary it can use to represent that dataset.

Even if we set the same methods and parameters, a tokenizer trained on an English text dataset will be different from another trained on a code dataset or a multilingual text dataset.

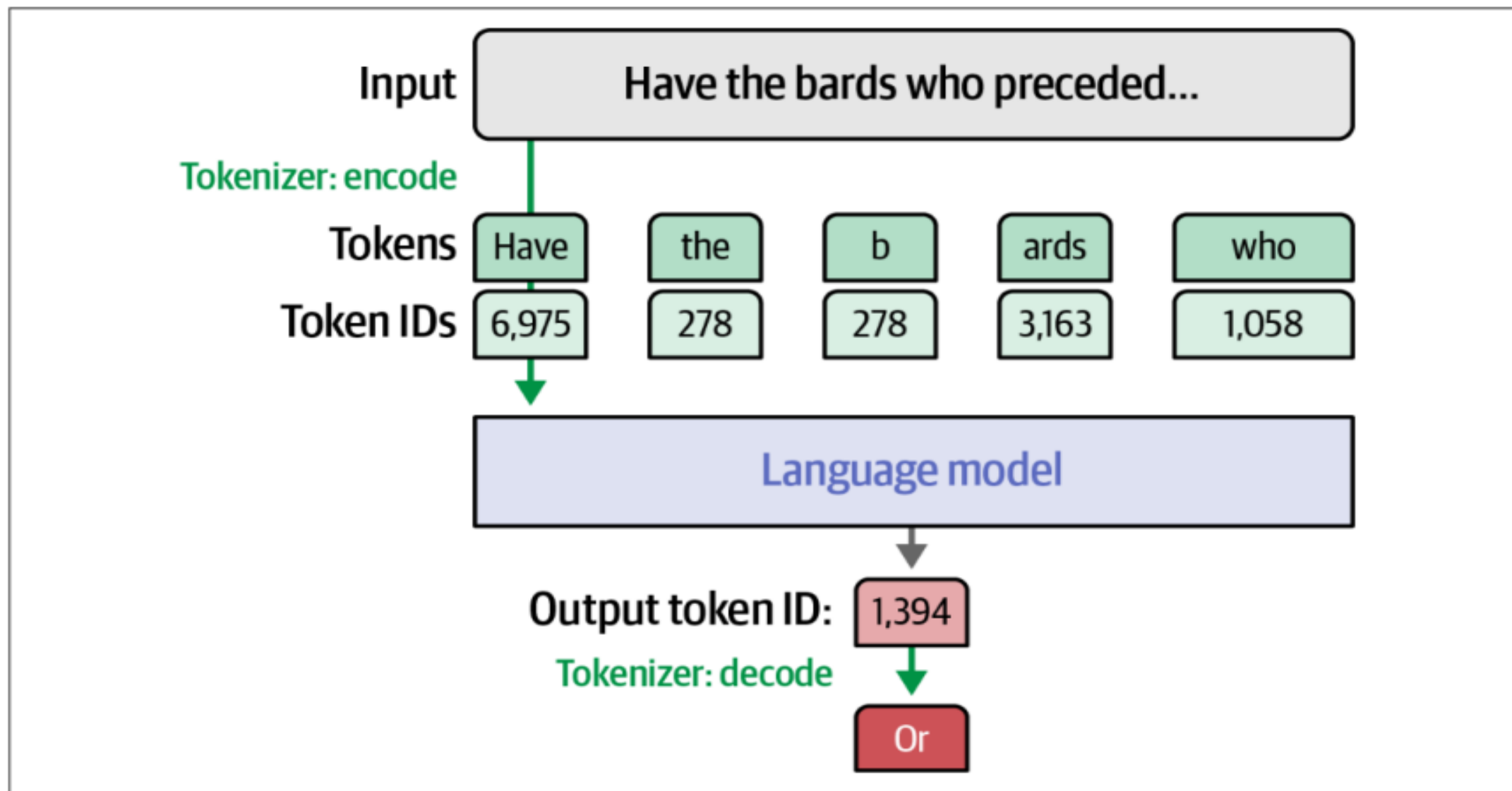


Figure 2-5. Tokenizers are also used to process the output of the model by converting the output token ID into the word or token associated with that ID.

Word Versus Subword Versus Character Versus Byte Tokens

Word tokens

- This approach was common with earlier methods like word2vec but is being used less and less in NLP. Its usefulness, however, led it to be used outside of NLP for use cases such as recommendation systems.
- One challenge with word tokenization is that the tokenizer may be unable to deal with new words that enter the dataset after the tokenizer was trained. This also results in a vocabulary that has a lot of tokens with minimal differences between them (e.g., apology, apologize, apologetic, apologist).
- This latter challenge is resolved by subword tokenization as it has a token for apolog, and then suffix tokens (e.g., -y, -ize, -etic, -ist) that are common with many other tokens, resulting in a more expressive vocabulary.

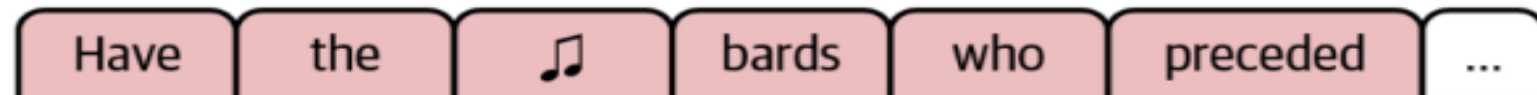
Subword tokens

This method contains full and partial words.

In addition to the vocabulary expressivity mentioned earlier, another benefit of the approach is its ability to represent new words by breaking down the new token into smaller characters, which tend to be a part of the vocabulary.

Text Have the 🎵 bards who preceded...

Word tokens



Subword tokens



Character tokens



H a v e <space> t h e <space> 🎵 <space>

Byte tokens

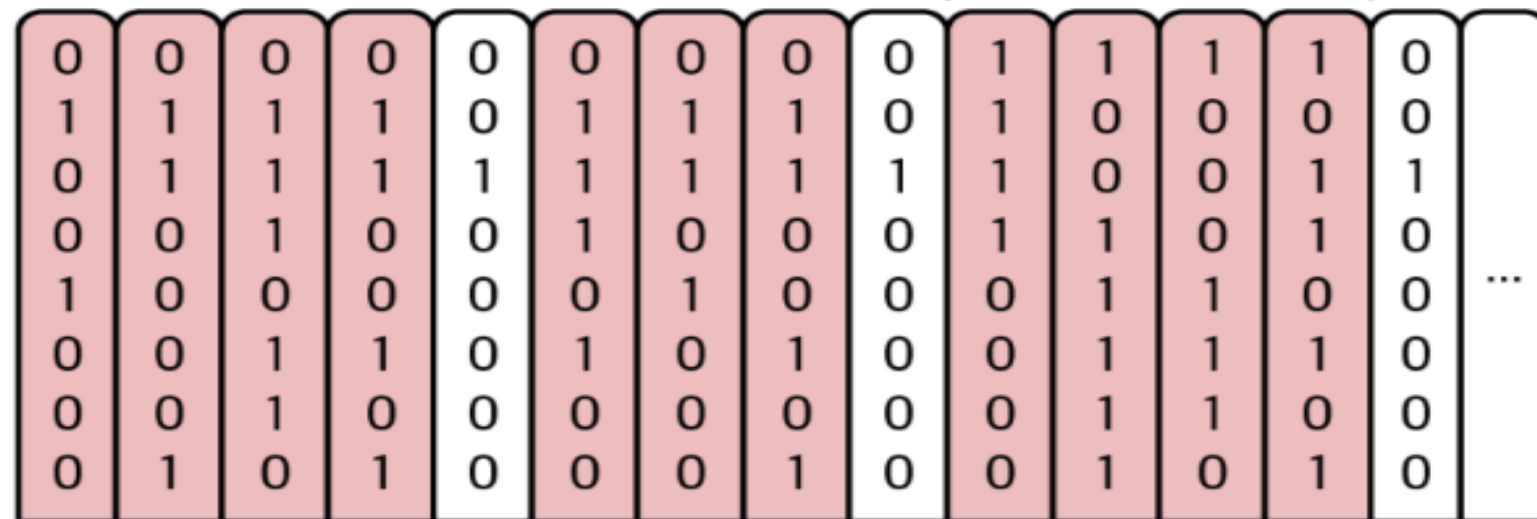


Figure 2-6. There are multiple methods of tokenization that break down the text to different sizes of components (words, subwords, characters, and bytes).

Character tokens

This is another method that can deal successfully with new words because it has the raw letters to fall back on. While that makes the representation easier to tokenize, it makes the modeling more difficult. Where a model with subword tokenization can represent “play” as one token, a model using character-level tokens needs to model the information to spell out “p-l-a-y” in addition to modeling the rest of the sequence.

Subword tokens present an advantage over character tokens in the ability to fit more text within the limited context length of a Transformer model. So with a model with a context length of 1,024, you may be able to fit about three times as much text using subword tokenization than using character tokens (subword tokens often average three characters per token).

Byte tokens

One additional tokenization method breaks down tokens into the individual bytes that are used to represent unicode characters.

Papers like “CANINE: Pre-training an efficient tokenization-free encoder for language representation” out-line methods like this, which are also called “tokenization-free encoding.”

Other works like “ByT5: Towards a token-free future with pre-trained byte-to-byte models” show that this can be a competitive method, especially in multilingual scenarios.

Tokenizer Properties

1.Tokenization methods

As we've seen, there are a number of tokenization methods with byte pair encoding (BPE) being the more popular one. Each of these methods outlines an algorithm for how to choose an appropriate set of tokens to represent a dataset.

```
from tokenizers import Tokenizer
from tokenizers.models import BPE
from tokenizers.trainers import BpeTrainer
from tokenizers.pre_tokenizers import Whitespace
bpe_tokenizer = Tokenizer(BPE(unk_token="[UNK]"))
bpe_tokenizer.pre_tokenizer = Whitespace()
trainer = BpeTrainer(
    vocab_size=100,
    min_frequency=1,
    special_tokens=["[UNK]", "[PAD]", "[CLS]", "[SEP]"]
)
```

```
corpus = [  
    "Machine learning is transforming healthcare with AI.", "Machine learning is  
widely used in healthcare.", "Artificial intelligence is transforming many  
industries."]  
bpe_tokenizer.train_from_iterator(corpus, trainer)  
encoding = bpe_tokenizer.encode(text)  
print("Tokens:")  
print(encoding.tokens)  
print("\nToken IDs:")  
print(encoding.ids)  
print("\nDecoded:")  
print(bpe_tokenizer.decode(encoding.ids))
```

Tokens:

```
['AI', 'is', 'ch', 'an', 'g', 'ing', 'healthcare', '.']
```

Token IDs:

```
[59, 31, 35, 30, 13, 28, 58, 4]
```

Decoded:

```
AI is ch an g ing healthcare .
```


2.Tokenizer parameters

After choosing a tokenization method, an LLM designer needs to make some decisions about the parameters of the tokenizer. These include:

Vocabulary size

How many tokens to keep in the tokenizer's vocabulary? (30K and 50K are often used as vocabulary size values, but more and more we're seeing larger sizes like 100K.)

Special tokens

What special tokens do we want the model to keep track of? We can add as many of these as we want, especially if we want to build an LLM for special use cases.

Common choices include:

- Beginning of text token <CLS>
- End of text token
- Padding token
- Unknown token
- Masking token

Capitalization

Token	Meaning	Purpose
[CLS]	Classification	Represents the complete sequence
[SEP]	Separator	Separates sentences/segments
[MASK]	Masking	Hides a token during training
[PAD]	Padding	Makes sequences equal length
[UNK]	Unknown	Represents an unknown token

[MASK] = "Hide this token and ask the model to predict it."

Original:

I love machine learning.

Masked:

I love [MASK] learning.

Prediction:

[MASK] → machine

3.The domain of the data

Even if we select the same method and parameters, tokenizer behavior will be different based on the dataset it was trained on (before we even start model training).

The tokenization methods mentioned previously work by optimizing the vocabulary to represent a specific dataset.

Raw Text



Tokenizer



Tokens



Token IDs



Embedding Layer



Transformer



Next-Token Prediction

Three important tokenizer properties for LLMs

1. Vocabulary size

A larger vocabulary can represent more words directly, but requires a larger embedding matrix.

2. Token efficiency

A good tokenizer represents text using a reasonable number of tokens.

"unhappiness"



["un", "happiness"]

instead of breaking it into many individual characters.

3. Special-token support

LLMs use special tokens to control processing:

[CLS] → classification

[SEP] → separation

[MASK] → masked prediction

[PAD] → padding

[UNK] → unknown token

- After a tokenizer is initialized and trained, it is then used in the training process of its associated language model.
- This is why a pre trained language model is linked with its tokenizer and can't use a different tokenizer without training.
- The language model holds an embedding vector for each token in the tokenizer vocabulary.
- When we download a pre trained language model, a portion of the model is this embeddings matrix holding all of these vectors.
- Before the beginning of the training process, these vectors are randomly initialized like the rest of the model's weights, but the training process assigns them the values that enable the useful behavior they're trained to perform.

Trained tokenizer

Tokens	
Token ID	Token
0	!
1	"
...	...
50,257	

Language model

Token embeddings	
0	
1	
...	...
50,257	

Figure 2-7. A language model holds an embedding vector associated with each token in its tokenizer.

Have the bards who preceded me left any theme unsung?



Contextualized token embedding vectors

Better token embedding vectors that incorporate more context

Figure 2-8. Language models produce contextualized token embeddings that improve on raw, static token embeddings.

Contextual Embeddings

Contextual embedding is a numerical vector representation of a word/token where its meaning is determined by the **surrounding context**.

Embedding = numerical representation of a token.

Contextual embedding = numerical representation whose value changes according to the surrounding context.

Have the bards who preceded me left any theme unsung?

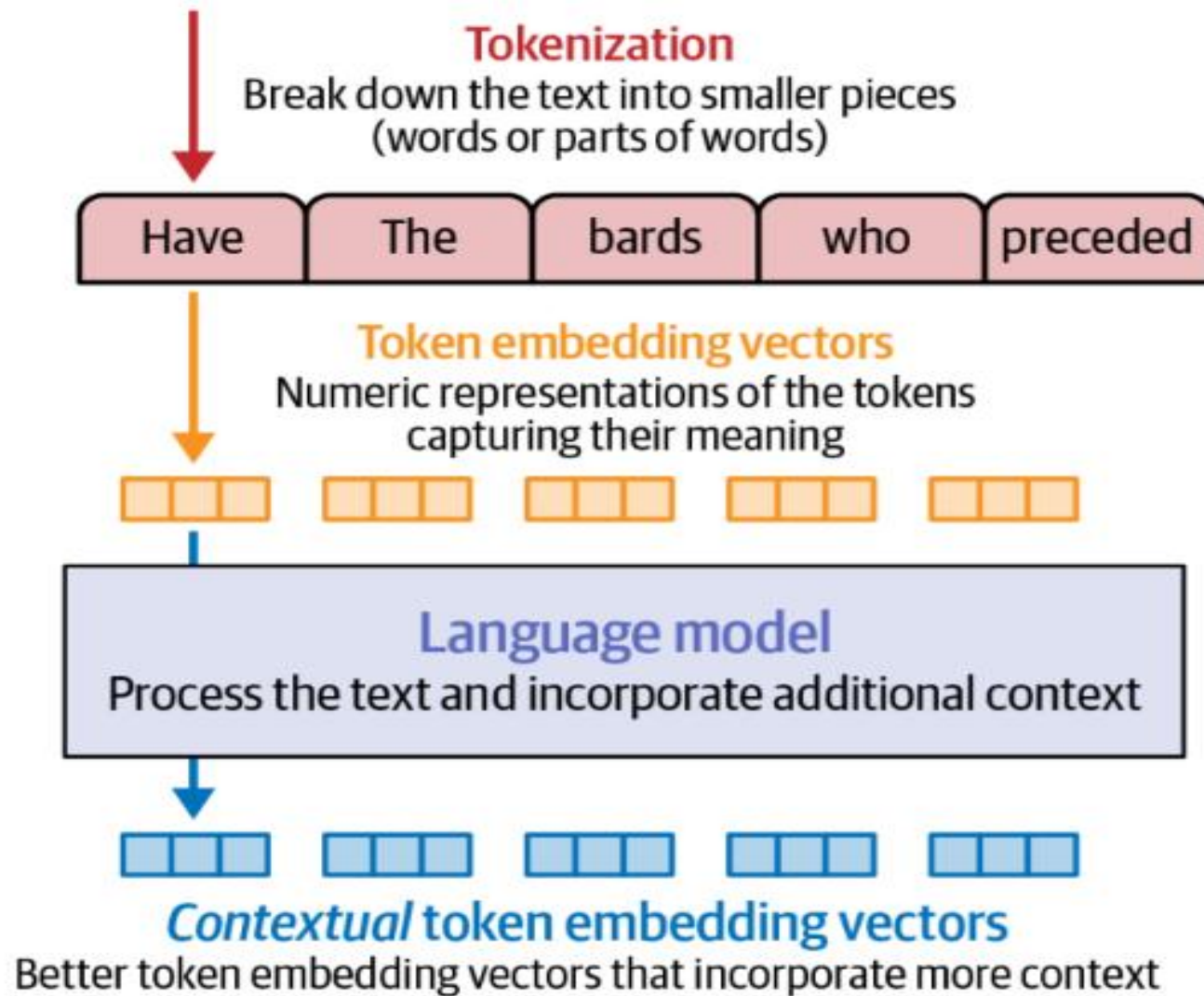


Figure 2-9. A language model operates on raw, static embeddings as its input and produces contextual text embeddings.

Feature	Static Embedding	Contextual Embedding
Example models	<i>Word2Vec, GloVe</i>	<i>BERT, RoBERTa</i>
Representation	<i>Fixed</i>	<i>Context-dependent</i>
bank in different sentences	<i>Same vector</i>	<i>Different vectors</i>
Handles ambiguity	<i>Limited</i>	<i>Better</i>
Uses surrounding words	<i>No</i>	<i>Yes</i>
Generated by Transformer	<i>No</i>	<i>Yes</i>

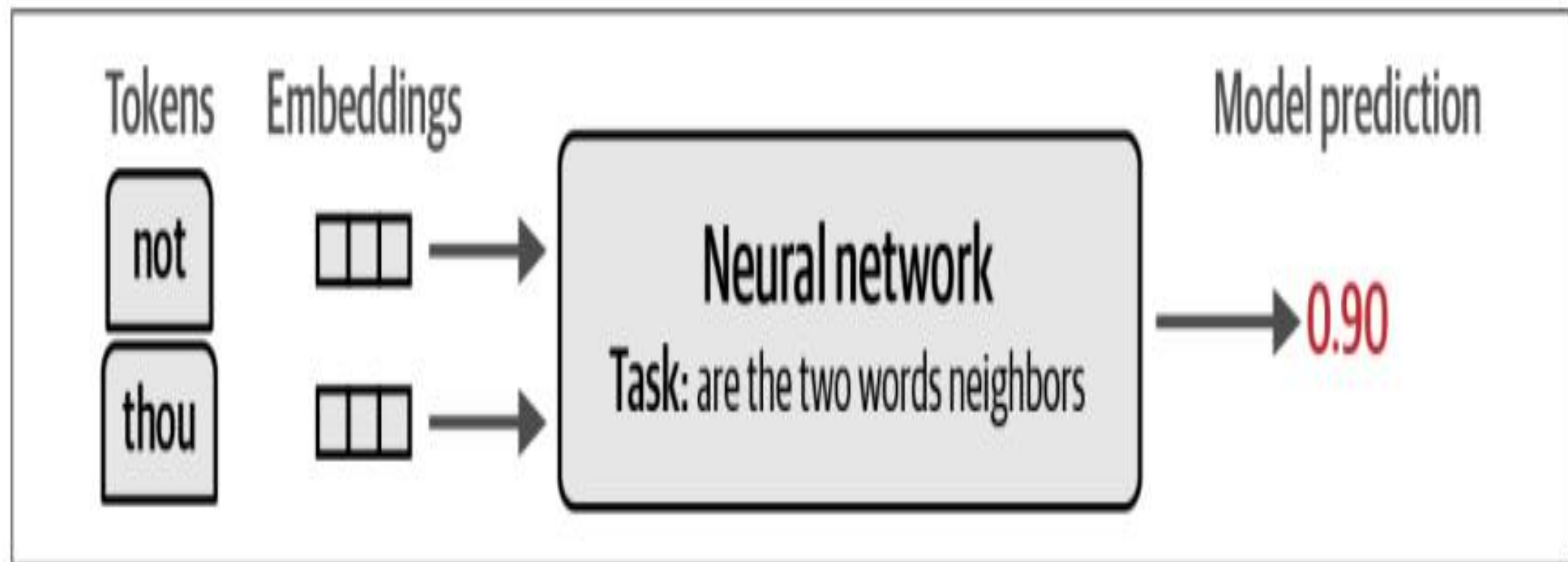


Figure 2-16. A neural network is trained to predict if two words are neighbors. It updates the embeddings in the training process to produce the final, trained embeddings.

```
import gensim.downloader as api

# Download embeddings (66MB, glove, trained on wikipedia, vector
size: 50)

# Other options include "word2vec-google-news-300"

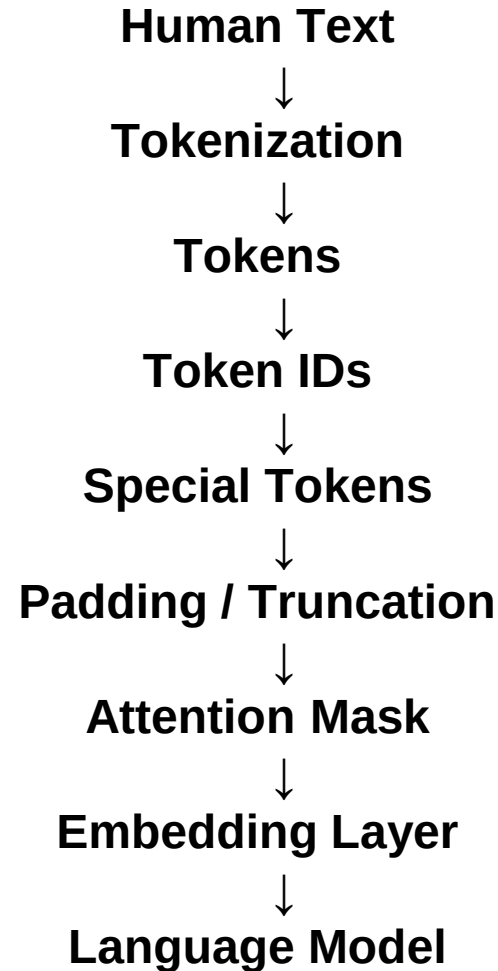
# More options at https://github.com/RaRe-Technologies/gensim-data

model = api.load("glove-wiki-gigaword-50")
model.most_similar([model['king']], topn=11)
```

```
[('king', 1.00000001192092896),  
 ('prince', 0.8236179351806641),  
 ('queen', 0.7839043140411377),  
 ('ii', 0.7746230363845825),  
 ('emperor', 0.7736247777938843),  
 ('son', 0.766719400882721),  
 ('uncle', 0.7627150416374207),  
 ('kingdom', 0.7542161345481873),  
 ('throne', 0.7539914846420288),  
 ('brother', 0.7492411136627197),  
 ('ruler', 0.7434253692626953)]
```


Feature	Word Embedding	Token Embedding	Text Embedding
Represents	<i>Word</i>	<i>Token</i>	<i>Sentence/document/text</i>
Example	<i>cat</i>	<i>cat, ing</i>	<i>"The cat is sleeping"</i>
Number of vectors	<i>One per word</i>	<i>One per token</i>	<i>Usually one per text</i>
Typical models	<i>Word2Vec, GloVe</i>	<i>BERT, GPT, LLMs</i>	<i>Sentence Transformers, embedding models</i>
Subwords	<i>Usually no</i>	<i>Yes</i>	<i>Not applicable at output level</i>
Main use	<i>Word meaning</i>	<i>Transformer input</i>	<i>Semantic similarity/search</i>
Context-dependent?	<i>Usually no</i>	<i>Can become contextual</i>	<i>Usually designed to capture whole-text meaning</i>

How Tokenizers Prepare Inputs to a Language Model



GPT Models

- GPT is a type of LLM.
- GPT is a Transformer-based autoregressive language model that is pre-trained to predict the next token and can therefore generate coherent sequences of text.
- The evolution of GPT models represents a transition from large-scale language generation to multimodal reasoning and agentic intelligence.
- Earlier models primarily focused on language understanding and generation, whereas newer models increasingly integrate advanced reasoning, long-context processing, tool use, coding, computer interaction, and autonomous multi-step task execution.

Step 1

Collect demonstration data and train a supervised policy.

A prompt is sample from our prompt dataset.

A labeler demonstrates the desired output behavior.

This data is used to fine-tune GPT-3.5 with supervised learning.



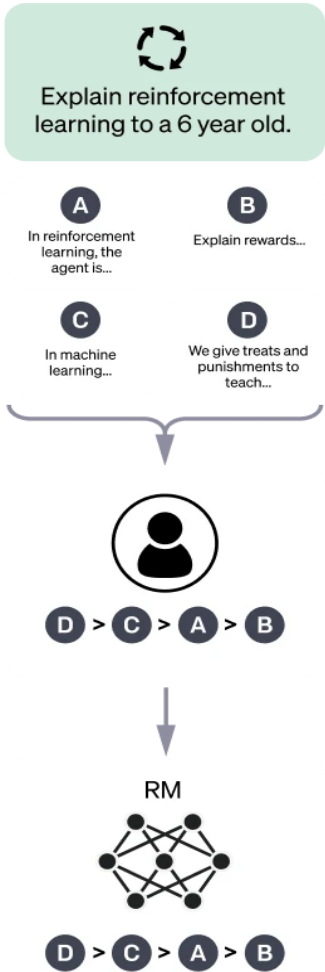
Step 2

Collect comparison data and train a reward model.

A prompt and several model outputs are sampled.

A labeler ranks the outputs from best to worst.

This data is used to train our reward model.



Step 3

Optimize a policy against the reward model using the PPO reinforcement learning algorithm.

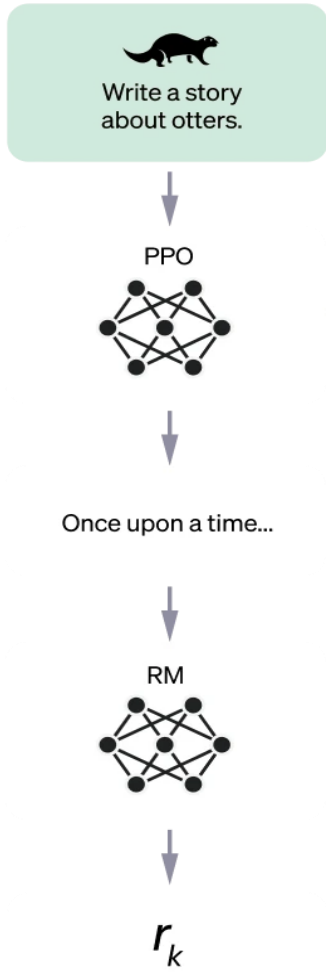
A new prompt is sampled from the dataset.

The PPO model is initialized from the supervised policy.

The policy generates an output.

The reward model calculates a reward for the output.

The reward is used to update the policy using PPO.

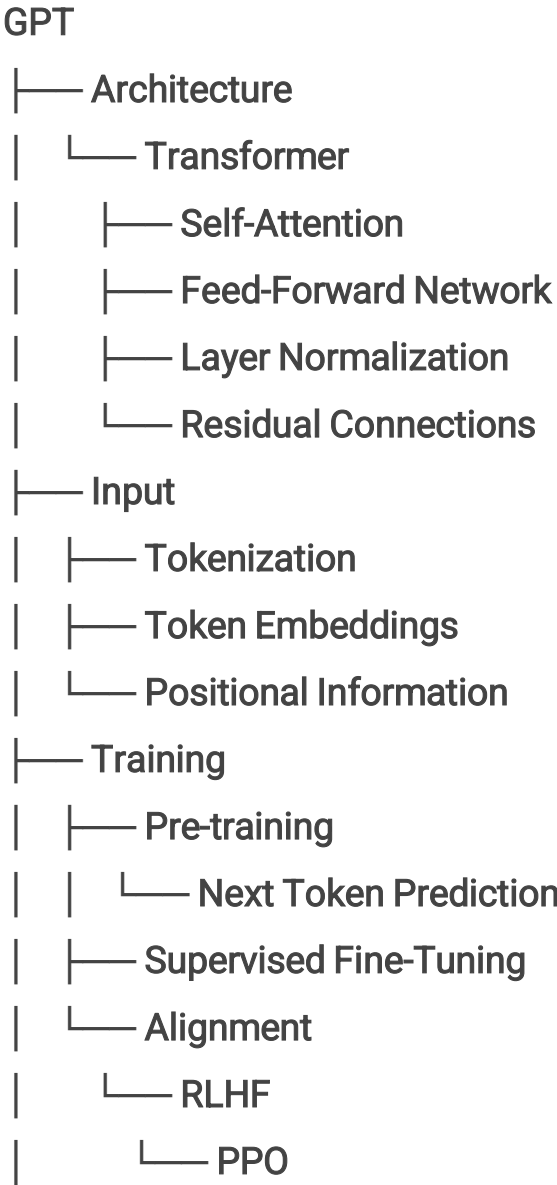


- **PPO (Proximal Policy Optimization)** has been used in the training of GPT-style models, particularly in **RLHF (Reinforcement Learning from Human Feedback)**.

GPT can be considered a **policy** in reinforcement learning:

- **State** → prompt/context
- **Action** → next token generated
- **Policy** → GPT model
- **Reward** → reward model's evaluation
- **Objective** → generate responses that receive higher rewards

GPT Concept Tree



Comparative Analysis of GPT Models

Model	Generation	Main strength	Reasoning	Speed	Context	Typical applications
GPT-3.5	GPT-3	Basic language generation	Low–Moderate	Fast	Smaller	Chatbots, simple Q&A, text generation
GPT-4	GPT-4	Strong reasoning & language understanding	High	Moderate	Large	Complex Q&A, analysis, coding
GPT-4o	GPT-4 generation	Multimodal interaction	High	Fast	Large	Text, image, audio, real-time interaction
GPT-5	GPT-5	Advanced reasoning	Very High	Moderate–Fast	Very large	Research, coding, analysis, agents
GPT-5.5	GPT-5 generation	Professional work & coding	Very High	Moderate	1.05M tokens	Software engineering, research, data analysis, computer-use tasks (OpenAI Developers)
GPT-5.6 Luna	GPT-5.6	Cost-efficient/high-volume AI	High	Very Fast	1.05M tokens	Education, everyday AI, high-volume applications (OpenAI Platform)
GPT-5.6 Terra	GPT-5.6	Intelligence–cost balance	Very High	Fast	1.05M tokens	General professional applications, agents (OpenAI Platform)
GPT-5.6 Sol	GPT-5.6	Complex professional reasoning	Very High	Moderate	1.05M tokens	Research, coding, cybersecurity, science, complex agents (OpenAI Developers)
GPT-6 Astra	GPT-6	Frontier reasoning & agentic work	Highest	Fast relative to capability	Very large	Computer use, software engineering, science, professional workflows (OpenAI Developers)